

Spatial Epidemiology

Network structure and spatial utilities in AlgebraicABMs.jl

Simon Frost

2026-03-30

1 Overview

AlgebraicABMs.jl provides tools for spatially-structured agent-based models:

1. **networkify** — automatically extends a schema with graph structure (vertices, edges, location homs)
2. **Spatial utilities** — `positions()`, `within_radius()`, `pairwise_distances()` for coordinate-based models
3. **shortest_distance** — BFS-based graph distance between vertices

This vignette demonstrates each capability with epidemiological examples.

2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

TaskLocalRNG()
```

3 Part 1: Network SIR with networkify

3.1 Defining the schema

We start with a simple schema where S, I, and R represent agents in each compartment, then use **networkify** to add graph structure.

```
@present SchSIR(FreeSchema) begin
    S::Ob
    I::Ob
    R::Ob
end
```

```
SchSIR_Net = networkify(SchSIR)
```

```
# Inspect the generated schema
gen_names = [first(g) for g in generators(SchSIR_Net)]
println("Objects: ", [first(g) for g in generators(SchSIR_Net, :Ob)])
println("Homs: ", [first(g) for g in generators(SchSIR_Net, :Hom)])
```

```
Objects: [:S, :I, :R, :V, :E]
Homs: [:src, :tgt, :loc_S, :loc_I, :loc_R]
```

networkify added:

- Objects V (vertices) and E (edges)
- Homs src, tgt : E → V (graph structure)
- Location homs loc_S : S → V, loc_I : I → V, loc_R : R → V (placing agents on the network)

3.2 Creating the ACSet type and initial state

```
@acset_type SIRNet(SchSIR_Net, part_type=BitSetParts)

# Build a small network: 8 nodes in a ring
n_nodes = 8
n_s = 6 # susceptible agents
n_i = 2 # infected agents

state = SIRNet()
add_parts!(state, :V, n_nodes)

# Ring edges (undirected = two directed edges per connection)
for i in 1:n_nodes
    j = mod1(i + 1, n_nodes)
    add_part!(state, :E; src=i, tgt=j)
    add_part!(state, :E; src=j, tgt=i)
end

# Place susceptible agents on nodes 1-6, infected on nodes 7-8
add_parts!(state, :S, n_s; loc_S=1:n_s)
add_parts!(state, :I, n_i; loc_I=(n_s+1):(n_s+n_i))

println("Network: $(nparts(state, :V)) nodes, $(nparts(state, :E)) edges")
println("S=$(nparts(state, :S)), I=$(nparts(state, :I)), R=$(nparts(state, :R))")

Network: 8 nodes, 16 edges
S=6, I=2, R=0
```

3.3 Infection rule: requires adjacency

Infection should only occur when a susceptible and an infected agent are on adjacent nodes. The rule pattern requires:

- An S agent on some node v1
- An I agent on some node v2
- An edge from v1 to v2

The rule converts the S agent to an I agent.

```
# Pattern L: S on v1, I on v2, edge v1→v2
L_inf = SIRNet()
add_parts!(L_inf, :V, 2)
add_part!(L_inf, :E; src=1, tgt=2)
add_part!(L_inf, :S; loc_S=1)
add_part!(L_inf, :I; loc_I=2)

# Interface I: keep v1, v2, edge, and I (S is consumed)
```

```

I_inf = SIRNet()
add_parts!(I_inf, :V, 2)
add_part!(I_inf, :E; src=1, tgt=2)
add_part!(I_inf, :I; loc_I=2)

# Right R: v1, v2, edge, old I stays, new I on v1 (converted from S)
R_inf = SIRNet()
add_parts!(R_inf, :V, 2)
add_part!(R_inf, :E; src=1, tgt=2)
add_part!(R_inf, :I; loc_I=2)
add_part!(R_inf, :I; loc_I=1)

l_inf = homomorphism(I_inf, L_inf; initial=(V=[1,2], E=[1], I=[1]))
r_inf = homomorphism(I_inf, R_inf; initial=(V=[1,2], E=[1], I=[1]))

infection = ABMRule(:Infection, Rule(l_inf, r_inf), ContinuousHazard(0.5))
println("Infection rule: S + I (adjacent) → I + I")

Infection rule: S + I (adjacent) → I + I

```

3.4 Recovery rule

Recovery converts an I agent to an R agent, independent of network structure.

```

# Pattern: single I agent on a node
L_rec = SIRNet()
add_part!(L_rec, :V)
add_part!(L_rec, :I; loc_I=1)

# Interface: just the node (I is removed)
I_rec = SIRNet()
add_part!(I_rec, :V)

# Right: node with R agent
R_rec = SIRNet()
add_part!(R_rec, :V)
add_part!(R_rec, :R; loc_R=1)

l_rec = homomorphism(I_rec, L_rec; initial=(V=[1],))
r_rec = homomorphism(I_rec, R_rec; initial=(V=[1],))

recovery = ABMRule(:Recovery, Rule(l_rec, r_rec), ContinuousHazard(Exponential(3.0)))
println("Recovery rule: I → R (mean time 3.0)")

Recovery rule: I → R (mean time 3.0)

```

3.5 Running the network SIR

```

abm = ABM([infection, recovery])
result = run!(abm, deepcopy(state); maxtime=25.0)

println("Events: $(length(result))")
for (i, (t, _, name, _)) in enumerate(result.events)
    i > 12 && break
    println("  t=$(round(t, digits=3)): $name")
end

```

```

# Final state
if !isempty(result.hist)
    final = codom(right(last(result.hist)))
    println("\nFinal: S=$(nparts(final, :S)), I=$(nparts(final, :I)), R=$(nparts(final, :R))")
end

[ Info: Stochastic scheduling algorithm ran out of events
Events: 14
t=0.106: Infection
t=0.168: Recovery
t=0.307: Infection
t=0.787: Infection
t=1.235: Infection
t=1.238: Recovery
t=1.246: Infection
t=1.258: Recovery
t=1.546: Infection
t=1.677: Recovery
t=2.451: Recovery
t=3.367: Recovery

```

Final: S=0, I=0, R=8

Infection spreads along the ring: only agents on adjacent nodes can interact. This is fundamentally different from a well-mixed model where any S-I pair can transmit.

4 Part 2: Spatial Utilities

For models with continuous spatial coordinates, AlgebraicABMs provides `positions()`, `within_radius()`, and `pairwise_distances()`.

4.1 Defining a spatial schema

```

@present SchSpatial(FreeSchema) begin
    Agent::Ob
    Coord::AttrType
    px::Attr(Agent, Coord)
    py::Attr(Agent, Coord)
end
@acset_type SpatialSet(SchSpatial, part_type=BitSetParts)

SpatialSet

```

4.2 Creating a spatial population

```

spatial_state = SpatialSet{Float64}()
add_parts!(spatial_state, :Agent, 6;
    px = [0.0, 1.0, 0.5, 5.0, 5.5, 10.0],
    py = [0.0, 0.0, 0.8, 5.0, 5.2, 10.0])

println("Agents placed at:")
for i in 1:nparts(spatial_state, :Agent)
    x = subpart(spatial_state, i, :px)
    y = subpart(spatial_state, i, :py)
end

```

```

    println(" Agent $i: ($x, $y)")
end

```

Agents placed at:

```

Agent 1: (0.0, 0.0)
Agent 2: (1.0, 0.0)
Agent 3: (0.5, 0.8)
Agent 4: (5.0, 5.0)
Agent 5: (5.5, 5.2)
Agent 6: (10.0, 10.0)

```

4.3 Extracting positions

```

pos = positions(spatial_state, :Agent, [:px, :py])
println("Position matrix (2 × $(size(pos, 2))):")
println("  x: ", pos[1, :])
println("  y: ", pos[2, :])

```

Position matrix (2 × 6):

```

x: [0.0, 1.0, 0.5, 5.0, 5.5, 10.0]
y: [0.0, 0.0, 0.8, 5.0, 5.2, 10.0]

```

4.4 Finding neighbors within radius

```

# Agents within distance 2.0 of Agent 1
near_1 = within_radius(spatial_state, 1, 2.0, :Agent, [:px, :py])
println("Agents within radius 2.0 of Agent 1: $near_1")

# Larger radius
near_1_wide = within_radius(spatial_state, 1, 6.0, :Agent, [:px, :py])
println("Agents within radius 6.0 of Agent 1: $near_1_wide")

# Include self
near_1_self = within_radius(spatial_state, 1, 2.0, :Agent, [:px, :py]; exclude_self=false)
println("Within radius 2.0 (including self): $near_1_self")

```

```

Agents within radius 2.0 of Agent 1: [2, 3]
Agents within radius 6.0 of Agent 1: [2, 3]
Within radius 2.0 (including self): [1, 2, 3]

```

Agents 2 and 3 are close to Agent 1; Agents 4-6 are far away. This can drive spatial transmission rules where only nearby agents interact.

4.5 Pairwise distance matrix

```

D = pairwise_distances(spatial_state, :Agent, [:px, :py])
println("Pairwise distance matrix ($(size(D))):")
for i in 1:size(D, 1)
    row = join([round(D[i, j], digits=2) for j in 1:size(D, 2)], " ")
    println("  [$row]")
end

```

Pairwise distance matrix ((6, 6)):

```

[0.0  1.0  0.94  7.07  7.57  14.14]
[1.0  0.0  0.94  6.4  6.88  13.45]
[0.94  0.94  0.0  6.16  6.66  13.22]
[7.07  6.4  6.16  0.0  0.54  7.07]

```

```
[7.57  6.88  6.66  0.54  0.0  6.58]
[14.14 13.45 13.22  7.07  6.58  0.0]
```

The distance matrix reveals spatial clusters: Agents 1–3 form one cluster, Agents 4–5 form another, and Agent 6 is isolated.

5 Part 3: Shortest Distance on Networks

For graph-based models, `shortest_distance` computes BFS shortest paths.

```
using Catlab.CategoricalAlgebra.CSets: MarkAsDeleted
@present SchNet(FreeSchema) begin
    V::Ob; E::Ob; src::Hom(E,V); tgt::Hom(E,V)
end
@acset_type NetGraph(SchNet, part_type=MarkAsDeleted)
NetGraph
```

5.1 A small network

```
# Linear chain: 1 → 2 → 3 → 4 → 5 (with back edges)
net = @acset NetGraph begin
    V = 5
    E = 8
    src = [1, 2, 3, 4, 2, 3, 4, 5]
    tgt = [2, 3, 4, 5, 1, 2, 3, 4]
end

println("Shortest distances from node 1:")
for v in 1:5
    d = shortest_distance(net, 1, v)
    d_str = d == typemax{Int} ? "∞" : string(d)
    println(" d(1, $v) = $d_str")
end
```

```
Shortest distances from node 1:
d(1, 1) = 0
d(1, 2) = 1
d(1, 3) = 2
d(1, 4) = 3
d(1, 5) = 4
```

5.2 Distance affects transmission probability

We can use graph distance in a `ClosureState` timer to make transmission probability decay with network distance.

```
# Two vertices connected by an edge - infection uses ClosureState
# that considers graph distance to modulate rate
L_net = @acset NetGraph begin V=2; E=1; src=[1]; tgt=[2] end
I_net = @acset NetGraph begin V=2 end
R_net = @acset NetGraph begin V=2 end

net_inf_rule = Rule(
    homomorphism(I_net, L_net; initial=(V=[1,2],)),
    homomorphism(I_net, R_net; initial=(V=[1,2],)))
```

```

# Rate decays with shortest distance between matched vertices
distance_timer = ClosureState(m -> begin
  state = codom(m)
  v1 = m[:V](1)
  v2 = m[:V](2)
  d = shortest_distance(state, v1, v2)
  d = d == typemax{Int} ? 100 : d
  Exponential(0.5 * (1 + d))
end)

net_infection = ABMRule(:NetInf, net_inf_rule, distance_timer)
abm_net = ABM([net_infection])

# Chain graph: 1-2-3-4-5 with all back-edges
init_net = @acset NetGraph begin
  V = 5
  E = 8
  src = [1,2,3,4,2,3,4,5]
  tgt = [2,3,4,5,1,2,3,4]
end

traj_net = run!(abm_net, deepcopy(init_net); maxtime=30.0)

println("Distance-dependent transmission:")
println("  Events: $(length(traj_net))")
for (i, (t, _, name, _)) in enumerate(traj_net.events)
  i > 8 && break
  println("    t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
Distance-dependent transmission:
Events: 8
t=0.046: NetInf
t=0.07: NetInf
t=0.092: NetInf
t=0.112: NetInf
t=0.134: NetInf
t=0.276: NetInf
t=0.72: NetInf
t=4.275: NetInf

```

Adjacent edges (distance 1) fire faster than distant ones, naturally producing wave-like spatial spread.

5.3 Disconnected components

```

# Two disconnected components
disconnected = @acset NetGraph begin
  V = 4
  E = 2
  src = [1, 2]
  tgt = [2, 1]
end

```

```

d12 = shortest_distance(disconnected, 1, 2)
d13 = shortest_distance(disconnected, 1, 3)
println("Connected pair: d(1,2) = $d12")
println("Disconnected pair: d(1,3) = $(d13 == typemax{Int} ? "∞ (unreachable)" : d13)")

```

```

Connected pair: d(1,2) = 1
Disconnected pair: d(1,3) = ∞ (unreachable)

```

6 Summary

Feature	Function	Use Case
Network schemas	<code>networkify(S)</code>	Add graph structure to any agent schema
Position extraction	<code>positions(state, ob, attrs)</code>	Get coordinate matrix for spatial agents
Neighbor search	<code>within_radius(state, i, r, ob, attrs)</code>	Find nearby agents for local transmission
Distance matrix	<code>pairwise_distances(state, ob, attrs)</code>	Full spatial distance computation
Graph distance	<code>shortest_distance(state, u, v)</code>	BFS distance on network topology

These tools enable models where **space matters**: disease spreads along network edges, transmission probability decays with distance, and interventions can target spatial clusters.